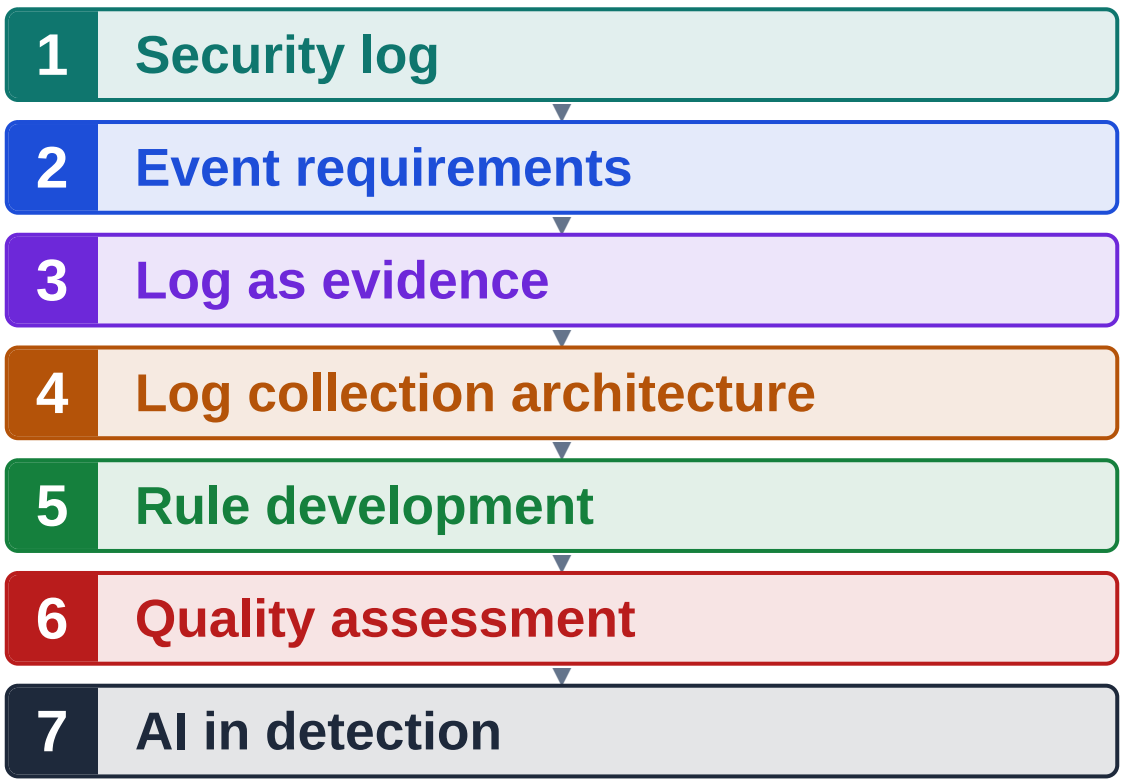
Lecture 3

Security Event Logging and Detection

Event requirements · evidential value · architecture · alert quality

Lecture outline



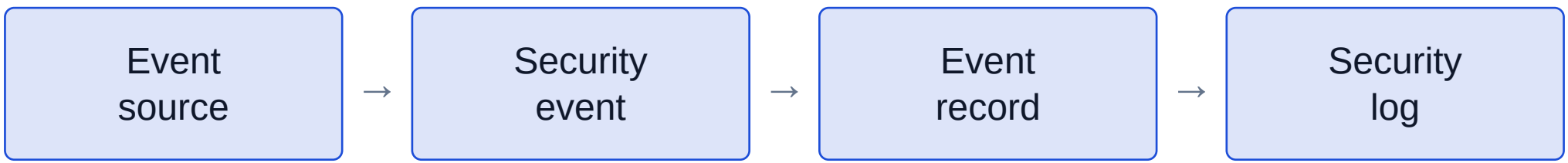
1

Security log

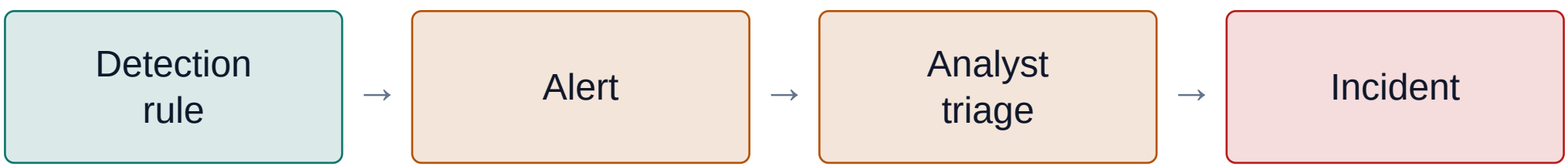
what we record and why

From event to incident

RECORDING



ANALYSIS



> An event only points to a **possible** violation — significance is established during analysis, not at recording time

One failure is normal, a series is an alert

	09:00	09:01	09:02	09:03	09:04	09:05	09:06	09:07
User			✗✓					
Attacker		✗✗✗	✗✗✗	✗✗✗	✗✗✗	✗✗✗	✗✗✗	✗✗✓

1 failure
a typo in the password — **normal**

vs

20 failures in 7 min
possible password guessing

An event's significance is determined **in the context** of other events — so routine actions are logged too

Security log ≠ operational log

	Operational	Security
Content	errors, response time, request execution flow	who did what to which object, with what result: logins, privilege changes, data access
Retention	days – weeks	months – years
Access	a wide range of engineers	restricted, separation of duties
Integrity	desirable	mandatory, changes are detectable

> Security event records go to a separate store with its own permissions and retention

Four purposes of a log

Detection	Investigation	Evidence	Data for models
all key events, low delivery latency	context, linking records across components, retention period	integrity, trustworthy time, attribution to a subject	a model cannot see what was not recorded

> Logging is a **design decision**: made together with the architecture, based on the threat model

Case: detection of SUNBURST, 2020

2FA event
a second device was added to an employee's account

→

Low severity
a routine operation with an authentication factor

→

Verification
an analyst calls the employee: "that's not my device"

→

Incident
a supply chain attack is uncovered

Operations on authentication factors are a **mandatory** event category

A low-severity event may be the **only** trace of an attack

From threats to an event catalog

Threat	What the attacker does	What the system records	Event type
Password guessing	tries passwords for someone else’s account	every login attempt; lockout after a series of failures	`auth.login` `auth.lockout`
SQL injection	enters a database command fragment into search	the firewall blocking the request	`waf.request.block`
Privilege escalation	tries to obtain administrator rights	role assignments, denied access to administration	`iam.role.assign` `authz.access`
Access to others’ data	puts someone else’s order number in the URL	access denied to the object	`authz.access`

> The system records **facts**; “password guessing” is already a **conclusion** of a detection rule

2

Event requirements

content · structure · prohibitions · mistakes

What to log

Authentication

login, logout, lockout

Authorization and privileges

access denial, role assignment

Protected data

viewing, export, deletion

Administration

configuration, secrets vault

Security errors

input, signature, token validation

Network and API

request rate, firewall blocks

Logging subsystem

agent stopped, records deleted

Weakening logging is a typical attacker step (ATT&CK T1562, T1070)

An event record answers questions

User **anna** tried to log in on 15 July at 11:23 MSK with a **wrong password**

When?	`"timestamp": "2026-07-15T08:23:45+00:00"`
What happened?	`"event_type": "auth.login"`
With what result?	`"outcome": "failure"`
Who?	`"actor": {"type": "user", "id": "anna"}`
From where?	`"source_ip": "203.0.113.42"`
Recorded by?	`"service": "auth-service"`
Details?	`"context": {"reason": "bad_password"}`

The result goes in a separate `outcome` field

Time in UTC: 08:23 UTC = 11:23 MSK

What must not be in a log

	✗ forbidden		✓ correct
1. Password	`"password": "qwerty123"`	→	`"reason": "bad_password"`
2. Token	`"token": "eyJhbGciOi…"`	→	`"token_hash": "3f9a1c"`
3. Card number	`"card": "4276 1600 1234 5678"`	→	`"card": "****5678"`
4. E-mail, phone	`"email": "anna@mail.example"`	→	`"actor": "anna"`
5. One-time code	`"otp": "482913"`	→	do not record — only the verification result

A password or token in a log lets anyone who reads the log log in as the user

Common mistakes and their cost

Unstructured records parsed with regular expressions	→	rules break when the text changes
Heterogeneous formats and field names	→	records cannot be matched
Records of different system parts are not linked	→	the chain of actions cannot be rebuilt
Only failures are logged	→	a successful attack is invisible
Mutable record storage	→	the attacker erases traces
Debug log level in production	→	data leakage, volume growth
Unsanitized user input (CWE-117)	→	forged records

Log4Shell: a string written to a log ran the attacker’s code on the server

International practices

OWASP ASVS 5.0 · V16
verifiable requirements: events, log protection, errors

OWASP Top 10:2025 · A09
logging and alerting failures: CWE-778, CWE-117, CWE-532

OWASP Logging Cheat Sheet
practical guidance on record content and protection

NIST SP 800-92
enterprise-scale log management

3

Log as evidence

STRIDE threat model

S	T	R	I	D	E
Spoofing	Tampering	Repudiation	Information disclosure	Denial of service	Elevation of privilege
authenticity	integrity	non-repudiation	confidentiality	availability	authorization

The log as evidence is the main control against **R**: a reliably recorded action is pointless to deny

Five properties of an evidential log

- **Complete recording**
- **Integrity**
- **Trustworthy time**
- **Attribution to a subject**
- **Preservation**

Hash chain

ORIGINAL CHAIN a record's hash is computed from its event and link; link = hash of the previous record

Record 1
login u-1042
link `0000`
hash `7c1e`

→

Record 2
role admin → u-2001
link `7c1e`
hash `5d20`

→

Record 3
report export
link `5d20`
hash `91ab`

→

Record 4
logout u-1042
link `91ab`
hash `c4f2`

AFTER TAMPERING WITH RECORD 2 the hash changed → the link in record 3 no longer matches

Record 1
login u-1042
link `0000`
hash `7c1e`

→

Record 2
role admin → **u-6666**
link `7c1e`
hash `e81b`

✗

Record 3
report export
link `5d20`
hash `91ab`

→

Record 4
logout u-1042
link `91ab`
hash `c4f2`

✓ **detected:** edits and deletions in the middle

✗ **missed:** tail truncation, full recomputation

so the last hash is periodically anchored **outside** the store

Retention periods



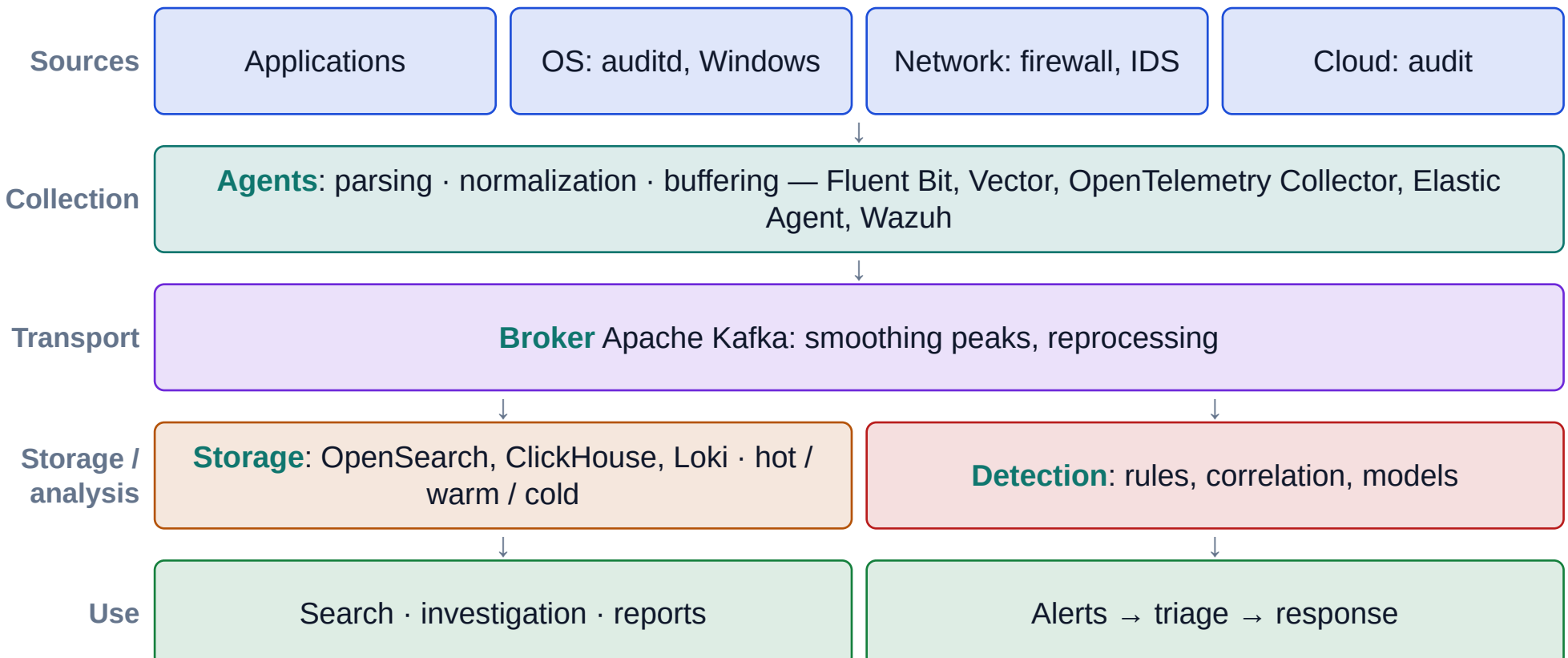
SUNBURST: from the attacker’s first actions to detection — **more than a year**

4

Log collection architecture

from source to analyst

Production log collection and analysis system



Key architectural decisions

Normalization at the source
rules are written against one schema: in-house, ECS / OpenTelemetry or OCSF

Message broker
storage failure ≠ record loss; new rules can be applied to the old stream

Stream processing
the rule is checked on every record — minimal latency

Scheduled processing
a query to the store — full history, but latency ≥ the run interval

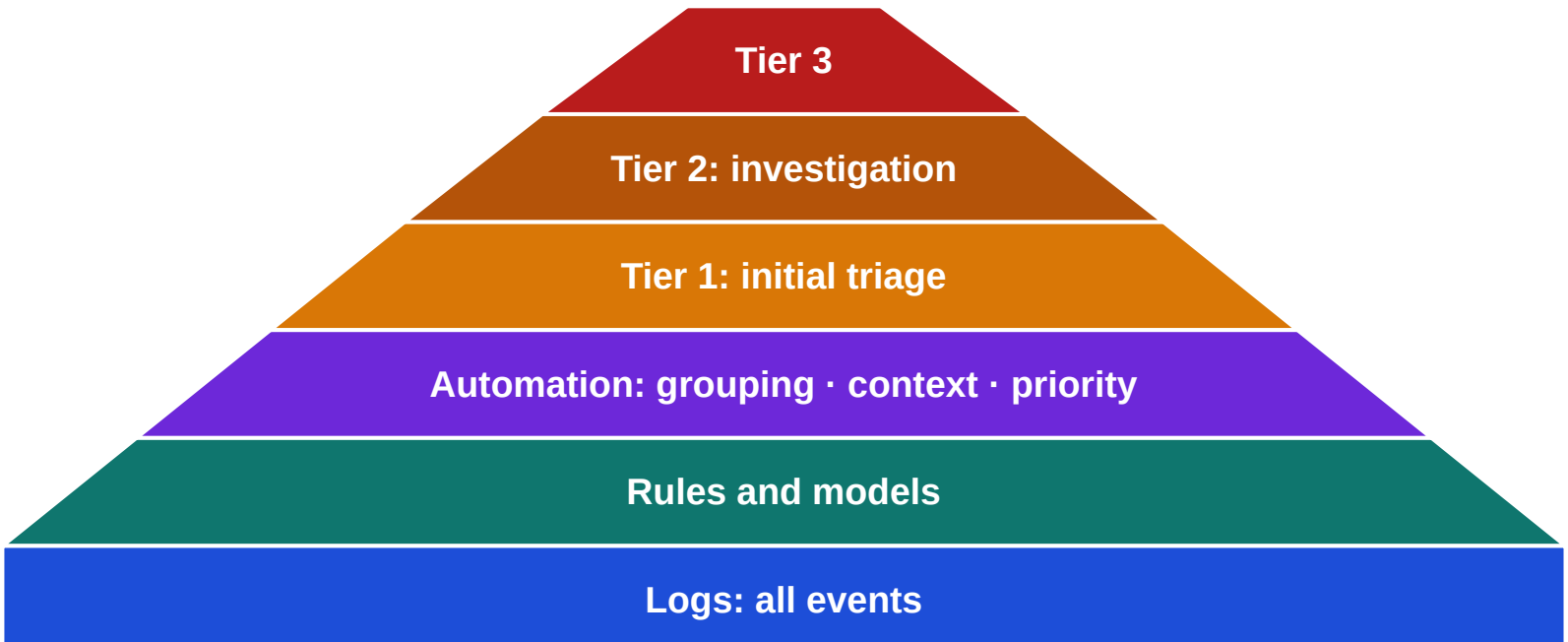
Open-source SIEMs compared

Solution	License	Rules and Sigma	Strength
Wazuh	**GPLv2 / Apache 2.0**	XML, 3000+ ready-made; Sigma since 5.0	agents: file integrity, configuration, vulnerabilities
OpenSearch Security Analytics	**Apache 2.0**	built-in Sigma, finding correlation	fully open stack
Elastic Security	**AGPL / SSPL / ELv2**	KQL, EQL, ES|QL; Sigma via pySigma	investigation; ML is paid
Grafana Loki	**AGPLv3**	LogQL; Sigma via pySigma	cheap storage, label index
Graylog Open	**SSPL**	streams and alerts	collection and parsing; security is paid
Security Onion	**ELv2**	Sigma, YARA, Suricata	ready-made platform with network sensors

OSI license **source available, service restrictions**

+ DFIR-IRIS (incidents), Shuffle (response)

Alert processing pyramid



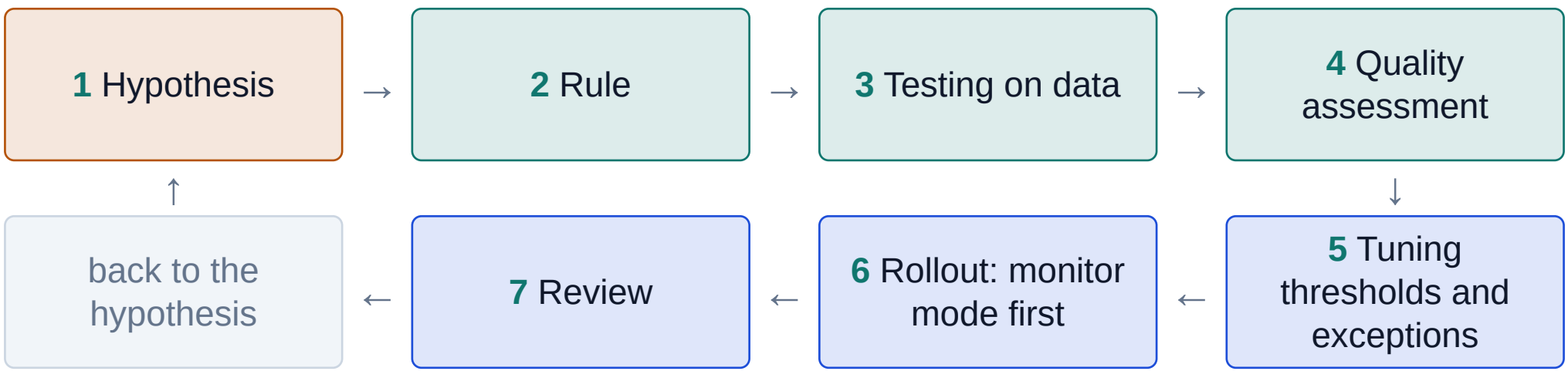
Feedback: every false positive filtered out by tier 1 is a reason to tune the rule

5

Rule development

hypothesis · windows · Sigma · ATT&CK

Rule life cycle



A rule is an engineering artifact: version, tests, quality metrics, owner

Detection hypothesis

Action	If the attacker does X	guesses a password (T1110.001)
Source	then in log S	of the authentication service
Trace	trace Z will appear	a series of failed `auth.login`
Distinction	distinguished by feature P	one account, short time span
Legitimate	but this also happens when W	forgotten password; an app with an old password

> The last line names **future false positives** in advance

Four kinds of rules

Single event

●

2FA disabled

no state needed

Window aggregation

●●●●●

5 failures in 10 min

counters per key

Event sequence

●●● → ●

failures → successful login

steps completed

Cross-source correlation

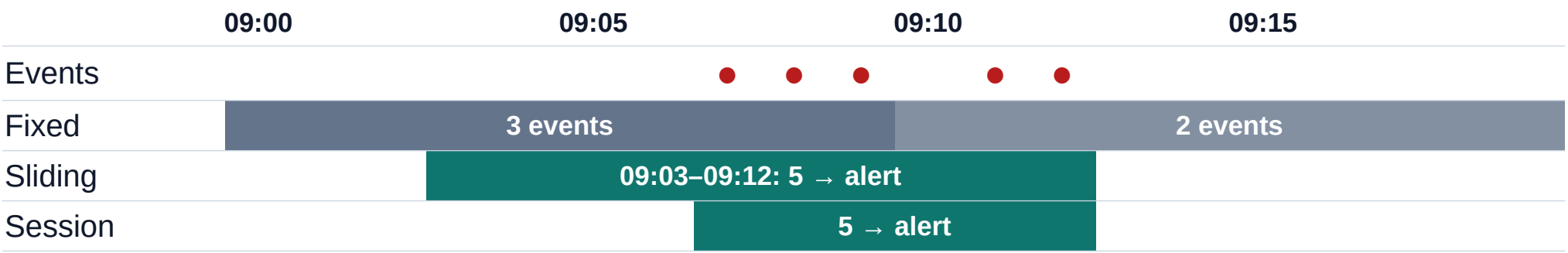
A + B + C

guessing → login → export

several streams

expressiveness, cost and dependence on data quality grow →

Aggregation window: where a series gets lost



window

counting interval

key

grouping field

threshold

value that triggers an alert

The grouping key decides what is visible

Key · measure	Guessing one account	Guessing from many addresses	Password spraying
Account · number of failures	✓	✓	✗
Address · number of failures	✓	✗	✓
Address · number of **distinct** accounts	✗	✗	✓

In SecureShop lockout stops guessing but not spraying — so the rule is needed precisely for spraying

A Sigma rule in plain terms

Task: “Tell me if someone fails to log in as **admin**”

```
title: Failed login as admin
```
Name

```
logsource: {service: auth}
```
Where to look? authentication log

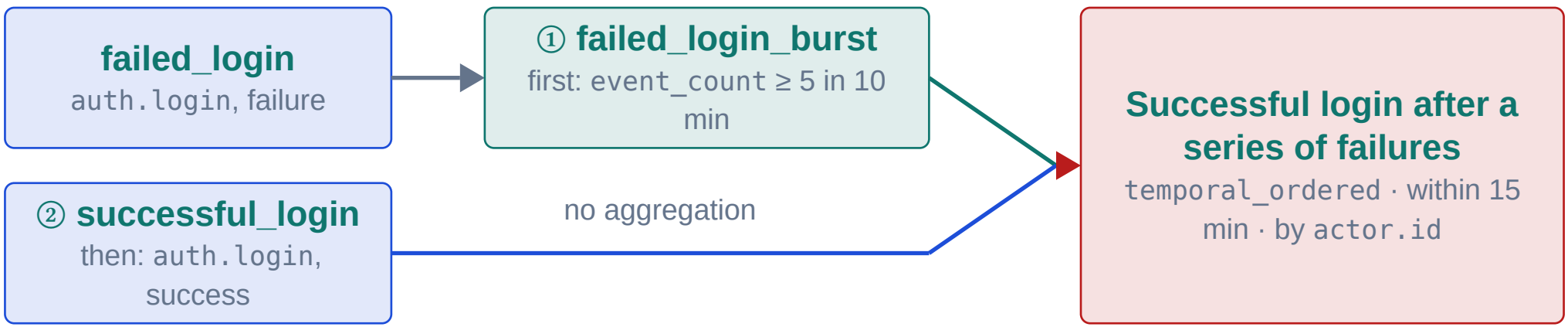
```
detection:
  selection:
    event_type: auth.login
    outcome: failure
    actor.id: admin
```
What to look for? all three conditions at once

```
  condition: selection
```
When to alert? when the conditions hold

```
level: medium
```
How important? medium severity

```
{"event_type": "auth.login", "outcome": "failure", "actor": {"id": "admin"}}
```
→ ✓ **alert**

Correlation in Sigma 2



An alert fires only if, for the same account, steps ① and ② occurred **in exactly this order** within 15 minutes

`event_count`	`value_count`	`temporal`	`temporal_ordered`
number of events	distinct values	all within the window	in a given order

Deprecated syntax: `condition: selection | count() > 5` with a `timeframe` field

Mapping to MITRE ATT&CK

Credential Access	T1110.001 guessing, T1110.003 spraying, T1556.006 2FA
Initial Access	T1078 valid accounts
Persistence / Privilege Escalation	T1098 account manipulation
Collection	T1213 data from information repositories
Defense Evasion	T1562 impair defenses

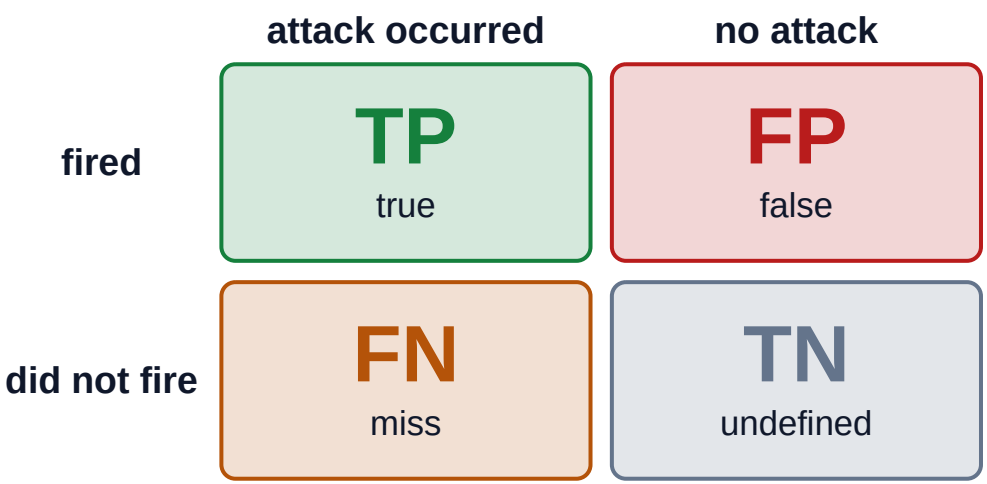
> Technique coverage ≠ one rule per technique: a rule sees a specific procedure, not the whole technique

6

Quality assessment

Precision · Recall · false positives

Confusion matrix and metrics



$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

share of useful alerts · counted **over alerts**

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

share of detected attacks · counted **over scenarios**

The base rate trap

10^6 events per day × 0.1 % false per event = ≈ 1000 FP per day vs ≈ 5 TP per day

Precision ≈ 5 / 1005 < 1 %

A low error rate per event does not yet make a rule usable

Reducing false positives — everything has a price

Threshold

higher threshold or longer series

price: a slow attack stays below the threshold

Exceptions

scanner, service accounts

price: an attacker acting as them is invisible

Grouping

repeats → one alert

price: the scale of the attack is lost

Context

role, host criticality, address reputation

price: dependence on reference data

Risk scoring

points per subject, alert on the total

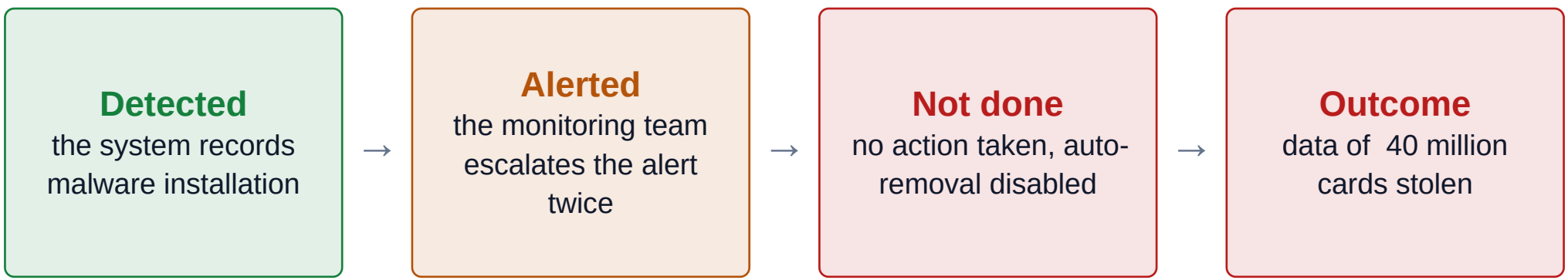
price: harder to explain the alert

Suppression

scheduled export, backup

price: the attacker mimics the process

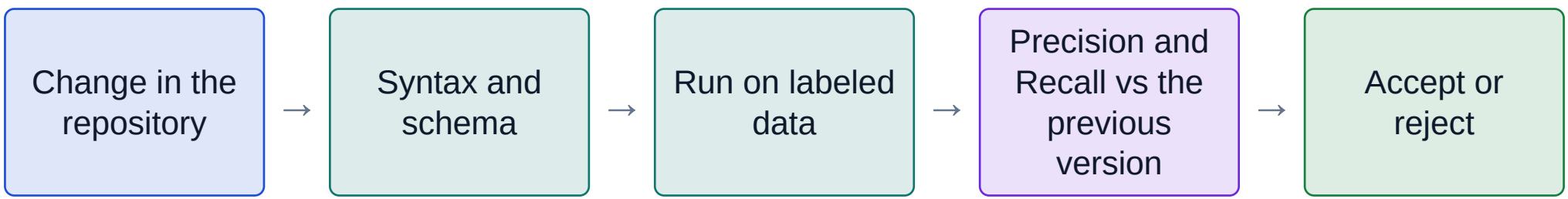
Case: Target, 2013



A rule without a response process does not protect

Disabling auto-response is risk acceptance — record it explicitly

Detection rules as code



Versions
history and rollback

Peer review
review of the change

Quality regression
degradation is not merged

7

Artificial intelligence in detection

where it is used · risks · quality

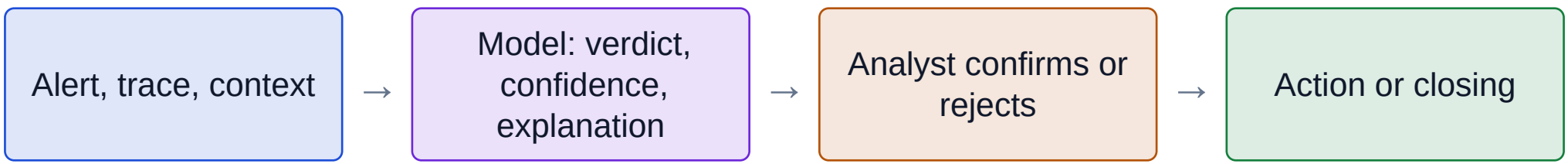
Where AI sits in the detection pipeline

Rules and models	anomalies in logs, behavior profiles (classic ML)	GuardDuty · Splunk ES
Automation	joins alerts into an attack chain, summary	Elastic Attack Discovery · GuardDuty ETD
1st line	an agent checks an alert: context, verdict, explanation	Google SecOps · MS Security Copilot · Datadog Bits AI · CrowdStrike Charlotte
3rd line	creates a rule from a threat description (YARA-L, ES|QL, SPL)	Google SecOps · Elastic AI Assistant · Splunk AI Assistant

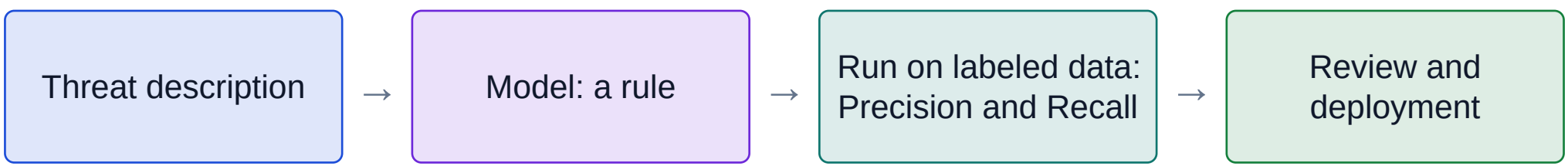
> AI is built on top of rules and the event store; it does not replace them

The model proposes — a person decides

ALERT CHECK



RULE CREATION



> A rule from a model goes through the same life cycle as a rule written by hand

Risks: a log is untrusted input for the model too

Instructions in log fields
User-Agent, page address, DNS query, user name — a text that tells the model what to do

Fields and techniques that do not exist
the model invents a schema field or maps ATT&CK wrongly

Feedback loop
the answer of the model goes to the same log and back to analysis

Cost and delay
every alert is a call to the model; select with rules first

68 %
of records suppressed by a role change

96 %
success when summarizing a log

11.8 %
best protection (was 26.6%)

Lesson of Log4Shell: log data is adversary input, for the AI analyst too

Quality of an AI analyst: a claim is not a measurement

Claimed by a vendor

> 98 %

agreement with the decisions of analysts of its own MDR service

not a check against truth; the share of misses is not given

Independent benchmark SecAlertBench

8322 records · 241 types · 16 models

recall 79.7% · F1 70.9%

false positives 44.1%

Accept a solution with a model like a rule: labeled data, Precision and Recall, watch mode, review

Key points

1 Security log	Not recorded — not detected
2 Event requirements	A common data format
3 Log as evidence	Completeness, integrity, trustworthy time, attribution, preservation
4 Collection architecture	Collection, transport, storage, detection, alert triage
5 Rule development	A rule turns a log from an archive into an attack detection tool
6 Quality assessment	Every rule must be evaluated: Precision and Recall
7 AI in detection	The model proposes, a person decides; a log is untrusted input for the model too

Lecture 3

Security Event Logging and Detection

Event requirements · evidential value · architecture · alert quality